

Bachelor's Thesis

Implementing Floating-Point Support in a Nanopass-Python Compiler

Marius Spill

Examiner: Prof. Dr. Peter Thiemann

Advisers: TO DO

Albert-Ludwigs-University Freiburg

Faculty of Engineering

Department of Computer Science

Chair for Programming Languages

September 15, 2026

Writing period

06.15.2026 – 09.15.2026

Examiner

Prof. Dr. Peter Thiemann

Advisers

TODO

Declaration

I hereby declare that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I hereby also declare that my Thesis has not been prepared for another examination or assignment, either wholly or excerpts thereof.

Place, Date

Signature

Abstract

foo bar

Contents

1	Introduction	1
1.1	Subchapter 1	1
1.2	Subchapter 2	1
2	Background	3
3	Design Architecture	5
3.1	Design Philosophy	5
3.2	Float Representation on the Heap	6
3.2.1	Alternatives	6
3.2.2	Possible Improvements	7
4	Implementation	9
4.1	Interpreter Extension and Phase-Driven Design	9
4.1.1	Syntactic Extensions (The Parser)	10
4.1.2	Static Semantics and the 64-Bit Precision Decision . .	10
4.1.3	Type-System Generalization	11
4.1.4	Interpreter Semantics	11
4.2	Extending the compiler to take literals through the whole pipeline	12
4.3	Heap Allocation for Floats	12
4.3.1	A Dictionary to Preserve Type Information	13
4.3.2	Handling Floats inside the Heap Allocator	13

4.4	Next section	14
5	Evaluation	15
6	Conclusion	17
	Bibliography	19

1 Introduction

1.1 Subchapter 1

The Hook: Why do we care about small/incremental compiler frameworks (like Nanopass)?

1.2 Subchapter 2

The Problem: What happens when a framework or downstream intermediate representation (IR) lacks native floating-point types? (e.g., restriction to purely integer arithmetic limits the computational expressiveness of the language). The Contribution: A brief high-level summary of what you added to the pipeline.

2 Background

The Nanopass Framework: Explain how the host compiler operates (small, discrete passes transforming highly specific AST micro-languages). Floating-Point Representation: Briefly introduce the target representation semantics (e.g., IEEE 754 standards) and how the target machine or runtime expects floats to be structured compared to integers.

3 Design Architecture

3.1 Design Philosophy

The governing principle of this extension was functionality first, optimisation second. This drove two central decisions. First, floats are represented as 64-bit doubles rather than 32-bit single-precision values, in order to preserve fidelity with Python’s native semantics. Second, floats are stored as boxed heap objects rather than as raw values, accepting the cost of indirection in exchange for reusing the compiler’s existing tuple and garbage-collection machinery. Performance optimisations were deferred until the base functionality for floats was complete, and were only revisited where time allowed.

A related decision concerns mixed-type arithmetic. Arithmetic is currently restricted to operands of identical type. The chosen design for lifting this restriction is implicit promotion: in any operation mixing an integer and a float, the integer operand is converted to a float before the operation is performed, so that the operation itself always acts on two floats. This keeps a single arithmetic path for floats and confines the type difference to a conversion step at the operands. The implementation of this promotion is pending.

3.2 Float Representation on the Heap

The garbage collector uses bit tagging to distinguish pointers, which it must follow, from raw values, which it must leave untouched. Integers are stored shifted one bit to the left, so that their least-significant bit (LSB) is 0, marking the word as a value the collector should not trace. Heap pointers instead carry an LSB of 1, signalling the collector to follow them. Every structure stored on the heap (tuples, closures, and so on) is laid out as a block of words prefixed by a header word.

A double in the IEEE-754 format cannot be tagged in this way, because all 64 bits are needed to encode the value, leaving no bit free for a tag. Worse, the bit that serves as the tag for integers corresponds, in a double, to part of the mantissa; repurposing it would distort the value entirely.

Floats must nonetheless be storable on the heap, since the language permits them as fields of objects, as function arguments, and as elements of tuples. Restricting the language to avoid this was rejected on principle, as the guiding philosophy stated above places functionality over performance. A representation compatible with the existing tagged collector was therefore required.

3.2.1 Alternatives

One option would have been to store floats as raw values directly on the heap. This would require overhauling the garbage collector to move away from bit tagging towards a layout map, that is, a description of the heap that records, for each structure, which words are pointers and which are raw values. Such a scheme would offer a scalable and memory-efficient way to represent any value directly on the heap, and would allow values to be stored inside any other structure without indirection. The drawback is that it conflicts with the architecture of the compiler as built so far, and that constructing and

maintaining layout maps would introduce considerable overhead.

The approach I chose instead was to box every float inside a two-word heap block: one header word and one word holding the value of the float. The header identifies the block as a boxed float. This yields a uniform representation that can be used for literals, for variables, and for floats nested in any heap structure, all through the same interface.

The intended behaviour is for the collector to recognise a boxed float from its header and to copy the payload word verbatim, without interpreting those raw bits as a pointer. As discussed in Section 3.2.2, this header-aware handling is not yet in place.

The advantage of boxing is that it offers a uniform design that delivers full functionality with little overhead and fits almost seamlessly into the existing compiler. The disadvantage is the inefficiency that arises from having to reach inside the box on every access to the value, and from allocating a fresh box for every float that is produced, which increases pressure on the collector. These costs were judged acceptable, since the trade-off aligned with the design philosophy of providing functionality before performance.

3.2.2 Possible Improvements

Efficiency could be improved by replacing bit tagging with a full layout or type map, which would permit storing raw floats directly on the heap rather than boxing them. Complementarily, the register allocator could be extended to distinguish floats from integers and to maintain a separate region of the stack for floating-point values, so that raw floats could be spilled to the stack just like integers, instead of being boxed and sent to the heap.

Independently of performance, the boxing scheme is not yet fully sound under the copying collector. After an object is copied, the collector scans each of its words and tests the LSB to decide whether the word is a pointer to follow. Because a boxed float's payload is a raw 64-bit double, a value whose

low bit happens to be 1 would be misinterpreted as a tagged pointer and erroneously followed. Making boxing correct therefore requires the header-aware traversal described above, so that the payload word of a float box is copied verbatim and never scanned as a pointer.

4 Implementation

The integration of floating-point operations requires a full-vertical extension across the compiler pipeline. This chapter details the engineering semantics, type systems, and lowering mechanisms required to support 64-bit double-precision floats within a base 32-bit architecture.

4.1 Interpreter Extension and Phase-Driven Design

Before implementing the concrete compiler backend, the language’s dynamic semantics were established by extending the reference interpreter. Because the interpreter defines the ground-truth behavioral specification of the language, this execution layer must be finalized first to ensure semantic parity with the compiler’s eventual target output.

To establish clear verification metrics, a differential test suite containing half a dozen Python programs was constructed. These test cases duplicated existing integer-based control flow, arithmetic expressions, and native built-in functions (such as `print()`), substituting integer literals with floating-point values. The immediate implementation goal was to allow the interpreter to execute this test suite natively without throwing exceptions, matching Python’s runtime behavior exactly.

Extending the interpreter required systematic modifications across three core subsystems of the pipeline: the abstract syntax tree (AST) parser, the static type-checker, and the evaluation semantics engine.

4.1.1 Syntactic Extensions (The Parser)

Extending the frontend parser was highly deterministic because the host language's `ast` module natively recognizes floating-point tokens. The internal AST grammar definition was upgraded to allow literal expression nodes (`EConst`) to hold float primitives alongside integers. Additionally, the type-mapping routines were extended to recognize and propagate explicit variable type annotations, allowing patterns such as `a : float = 3.14` to pass successfully through the front-end token stream.

4.1.2 Static Semantics and the 64-Bit Precision Decision

The primary design challenge emerged during the static analysis phase within the type-checking subsystem. Because the compiler targets a 32-bit RISC-V architecture (RV32), implementing 32-bit single-precision floats (the RISC-V "F" extension) would have offered significant implementation expediency; 32-bit floats map cleanly to standard 32-bit stack slots and integer register pairs.

However, architectural integrity took precedence over implementation simplicity. Python natively treats all floating-point numbers as 64-bit doubles. Forcing a static 32-bit truncation at the type-checker level would introduce severe semantic drift from standard Python behavior, causing silent precision loss on common practical decimals like `0.1` or `3.14`.

A hybrid architecture—selectively flagging exact base-2 powers (e.g., $0.5 = 2^{-1}$) as 32-bit floats and fallback decimals as 64-bit doubles—was also explored and ultimately rejected. Python does not natively support or expose mixed-precision literals; forcing such an abstraction would create an un-Pythonic dialect. Furthermore, it would introduce significant complexity into the backend code generator, requiring exhaustive permutations of type-coercion routines and hardware conversion instructions for cross-type arithmetic operations. To preserve strict language fidelity, a unified 64-bit

double-precision model was selected, targeting the RISC-V Double-Precision ("D") extension. Consequently, the type validator was designed to enforce a single constraint: verifying that literals do not exceed the 64-bit IEEE 754 infinity threshold.

4.1.3 Type-System Generalization

Beyond validating literals, the type-checker's expression evaluation routines needed a clean refactoring. Originally, the compiler assumed almost everything was an integer, evaluating binary operators and functions like `print()` by checking them directly against a hardcoded `TInt()` type. To support floating-point numbers without copying and pasting the same validation logic everywhere, this rigid approach was replaced with a more flexible system. A new helper function, `check_types_supported`, was introduced to validate expressions against a list of acceptable types instead of just one. For this first iteration, operands must still match exactly—meaning mixed-type operations like `float + int` are blocked for now. However, this refactor ensures that expressions evaluate to and return their correct type primitive (like `TFloat()`) rather than defaulting to an integer, laying the clean groundwork for future cross-type arithmetic.

4.1.4 Interpreter Semantics

Extending the runtime behavior in the interpreter was the most straightforward step because the interpreter itself is written in Python. Since the host language already supports 64-bit floating-point math natively.

In `semantics.py`, the evaluation rules for binary expressions were simply updated to accept float values alongside integers. When the interpreter encounters an operation like addition, it handles the AST nodes by passing the raw values directly to Python's native operators. By leveraging the host language's execution environment this way, the interpreter was immediately

able to run the floating-point test suite with perfect accuracy.

4.2 Extending the compiler to take literals through the whole pipeline

- `pass_1_2_shrink`: Had to add a 64-bit flag if constant is float so down the line its exact ieee structure isnt lost.
- `pass_1_2_shrink`: Else the pass didnt have to be adjusted for floats (maybe only the ast adding float)

4.3 Heap Allocation for Floats

The decision to box every float carried two practical consequences for the allocation pass. From this pass onwards, a float is represented as a pointer to its box, so any operation that consumes a float must first unbox it to recover the underlying value, and any operation that produces a float must box the result again before it can be stored back on the heap. The difficulty is that the allocation pass must know, for each operand, whether it is a float; otherwise it would apply the arithmetic to the box pointer rather than to the value it points to. The type information needed to make this decision was computed earlier, by the type checker, so the task was to make that information available at this point in the pipeline and to consult it for each operand.

The first step was a function that determines whether a given expression yields a float. Whenever it does, the uniform boxing design dictates that the operand must be unboxed before the operation and, where the operation itself yields a float, boxed again afterwards.

A complication arose for values held inside tuples (and for closures, functions, and objects, which are all represented as tuples at this stage).

The type checker does verify the type of every element of a tuple, but that per-element information was not preserved beyond type checking, so by the time the allocation pass ran there was no way to recover whether the element at a given position was a float. The expression's shape alone is sufficient for literals and for nested arithmetic, but not for accesses into a tuple or for plain variables, where the type is not visible in the expression itself. To resolve this, I returned to the type checker and preserved the type information it had already derived.

4.3.1 A Dictionary to Preserve Type Information

The result was a dictionary that, for each such structure, uses its identifier as the key and stores its fields or elements together with their corresponding types as the value. This dictionary is returned from the type checker to the main compiler driver, stored there, and passed on to every pass that requires it.

Up to this point in the pipeline, two passes consume the dictionary. The first is `uniquify`, which renames variables and must therefore update the dictionary's keys so that the recorded types remain reachable under the new names. The second is the heap allocator, for which the dictionary was originally introduced, where it is used to decide whether a given element is a float.

4.3.2 Handling Floats inside the Heap Allocator

When one or more operands of an operation are floats, the allocator emits a block that first extracts each float value from its box, then performs the operation on the unboxed values, and finally, where the result is itself a float, stores it back into a freshly allocated box. When none of the operands are floats, the operation is emitted unchanged, exactly as before.

4.4 Next section

To be implemented !

5 Evaluation

Correctness Validation: Instead of standard benchmarks or accuracy percentages, compiler evaluations validate determinism and compliance. Explain your test suite framework. Test Cases: Discuss how you validated corner cases (e.g., parsing precision limits, type coercion, complex floating-point expressions). Performance (Optional): If you optimize passes or look at compilation times, present it here. Otherwise, focus heavily on functional correctness and compilation validation.

6 Conclusion

Short summary of your execution and a couple of bullet points on future work (e.g., supporting double-precision, floating-point optimization passes, or array allocations of floating-point numbers).

Bibliography

